



Xi'an Jiaotong-Liverpool University

西交利物浦大學

MFE113TC

Programming Foundation for Intelligent Manufacturing Engineering

Lecture 06 Array & Pointer

Dr. Jiachen Liu

jiachen.liu@xjtlu.edu.cn

Fall, 2026

Outline

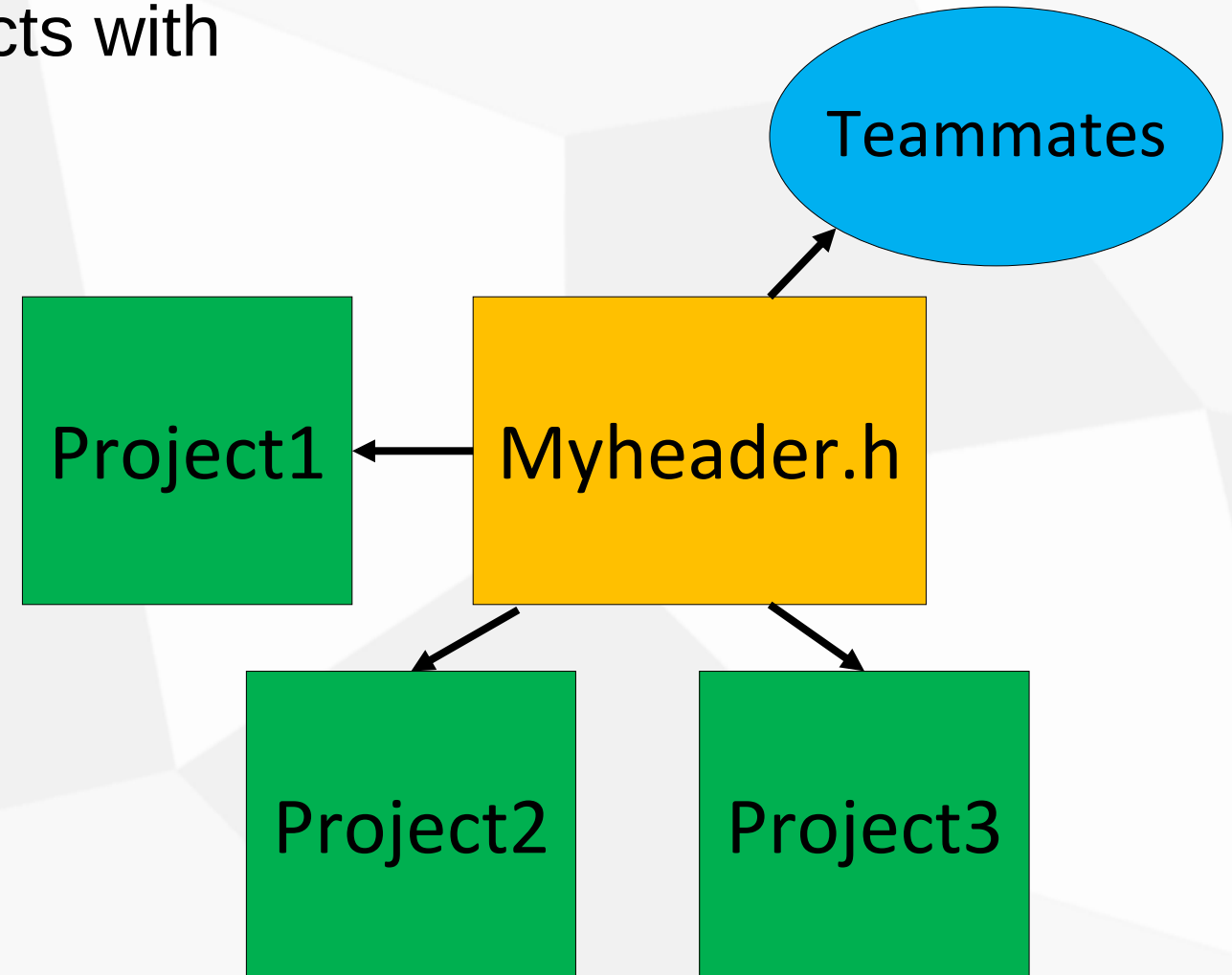
- How to create your own header file;
- Compile multiple files;
- Array;
- Multi-dimensional array;
- What is a pointer;
- Pointer & Array;

How to Create Your Own Header File?

- We frequently utilize official header files like `<stdio.h>` and `<iostream>`, they equip us with functions tailored for specific tasks.
- We've also learned how to define functions by ourselves.
- So, **what comes next? Let's learn to create our own header files**

How to Create Your Own Header File?

- Why we need to split code into **multiple source files**? This strategy facilitates the management of large-scale projects with numerous files.
- Reuse code even in different projects.
- Ease of Maintenance: make the update and bug fix easily.



How to Create Your Own Header File?

Prototype

```
#include <stdio.h>
```

```
void self_introduce();  
void draw_map();
```

```
int main()  
{  
    self_introduce();  
    draw_map();  
    return 0;  
}
```

Definition

```
void self_introduce()  
{  
    printf("Hello\n");  
    char name[] = "James";  
    printf("My name is %s\n",name);  
    int age = 19;  
    printf("I'm %d years old\n",age);  
    printf("I'm a rookie in programming.\n");  
    printf("I can draw a simple map in C.\n");  
}
```

```
void draw_map()  
{  
    printf("*****\n");  
    printf("*S          *\n");  
    printf("* ***** *\n");  
    printf("* ***** *\n");  
    printf("*          E*\n");  
    printf("*****\n");  
}
```

02

C myHeader.c

Source

C myHeader.h

Header

C test.c

- The test.c file can be simplified as:

```
#include <stdio.h>  
#include "myHeader.h"
```

```
int main()  
{  
    self_introduce();  
    draw_map();  
  
    return 0;  
}
```

Compile multiple files

- We can still utilize the **command line** to compile our program in the **terminal**. However, with multiple source files now involved, the compilation command needs to be adjusted as follows:

`gcc -o test test.c myHeader.c` Or `gcc test.c myHeader.c -o test.exe`

- Or if we have additional source files and prefer not to list their names individually, we can then use:

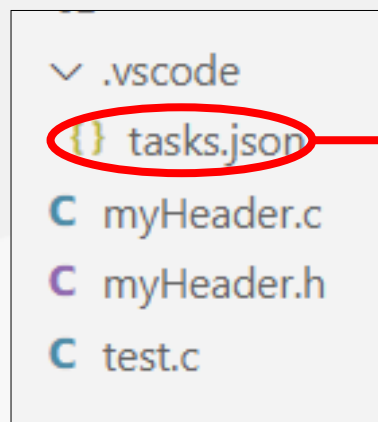
`gcc -o test *.c` Or `gcc *.c -o test.exe`

- This command will compile **all .c files** located in the same directory as the terminal's current path

Compile multiple files

- Additionally, we can use VSCode to compile multiple files, but it requires modifying the **tasks.json** file. Replace the `"${file}"` with the following:

```
"-I${workspaceFolder}",  
"${workspaceFolder}/*.c",
```



```
"tasks": [  
  {  
    "type": "cppbuild",  
    "label": "C/C++: gcc.exe build active file",  
    "command": "D:\\software\\mingw\\mingw64\\bin\\gcc.exe",  
    "args": [  
      "-fdiagnostics-color=always",  
      "-g",  
      // "${file}",  
      "-I${workspaceFolder}", // Link header files.  
      "${workspaceFolder}/*.c", // Link .c source files.  
      "-o",  
      "${fileDirname}\\${fileBasenameNoExtension}.exe"  
    ],  
    "options": {  
      "cwd": "${fileDirname}"  
    },  
  },  
]
```


Compile multiple files

- **Exercise: Function Declaration and Definition (Important!)**
- **Declare and Define a Function:**
 - Create a function that prints your name to the terminal.
- **Split the Function into Header and Source Files:**
 - Practice separating your function into a .h file for declaration and a .c file for definition.
- **Compile the Program:**
 - Attempt to compile your program using both methods you've learned.

Array

- In C programming, an array is essentially a group of elements that belongs to the **same type**, meanwhile with the **same name**;
- Array elements are arranged **sequentially in the memory space**, but you can access them in any order you want;
- Each element of an array is accessed using an **index** (an integer). The **index ranges from 0 to N-1**, where N is the array length;
- Array are commonly linked to pointers.

Array Index

- Consider a 10-element array of integers named `haha_array`;

`int x_array[10];`

- `x_array[0]`: refers to the name of the 1st array element;
- `x_array[9]`: refers to the name of the 10th array element;
- `x_array[10]`: is outside of the array bounds;
 - This calling **may cause runtime error** depending on the program;
- * Try it in Matlab and see what is the difference.**

my_array[0]	my_array[1]	my_array[2]	my_array[3]	my_array[4]	my_array[5]	my_array[6]	my_array[7]	my_array[8]	my_array[9]
-------------	-------------	-------------	-------------	-------------	-------------	-------------	-------------	-------------	-------------

Declaring and Initializing Arrays

- Arrays occupy space in the computer memory system;
- We need to specify the **type of each array element** and the **number** of these elements.

```
int my_array[10];  
char x[100], y[20];
```

- Arrays can also be initialized with a loop or at declarations:

```
for(i=0;i<10;i++)  
{  
    my_array[i]=0;  
}
```

```
int array[10]={0};
```

Declaring and Initializing Arrays

- Arrays can be declared and initialized at the same time with the help of initializers;



```
int my_array[10]={1,2,3,4,5,6,7,8,9,10};  
float a[5]={1.0,2.0,3.0,4.0}; /* remaining elements are initialized to 0 automatically */  
int n[]={1,2,3,4,5}; /* array size is omitted */
```

- The following statement will cause a syntax error. **Why???**



```
double x[5] = {3.0,5.0,7.0,4.0,5.0,9.0};
```

Declaring and Initializing Arrays

- Can we initialize an element at the middle of an array without giving values to all the preceding elements?
 - No! We cannot. Initialization of elements must begin at element 0 (1st).

```
int a[4] = {,,,6}; // Invalid, cannot initialize it this way.  
int a[4] = {6}; // Valid, begin at element 0 (1st)
```

- An alternative method:

```
int a[4];  
a[3] = 6;
```

Iterating through Array Elements

- A natural way to **iterate** through all the elements of an array is to use a **for** loop:

```
int my_array[10]={1,2,3,4,5,6,7,8,9,10};  
print("List all of the array elements");  
for(m=0;m<10;m++)  
{  
    printf("%d \t", my_array[m]);  
}
```

- It is incorrect to access array elements outside the declared array length. However, whether there will be an error prompted depending on the compiler you are using. **So Be Careful.**

Arrays and Strings

- Character arrays can be initialized by individual characters:

```
char text[]={'f','o','c','u','s','!'};
```

- They can also be initialized with a string:

```
char text[]="focus!";
```

- Remember a string is a special character array ending in the NULL character '\0'.
 - It is equivalent to: **char** text[]={'f','o','c','u','s','!','\0'};

Quick Quiz about Arrays

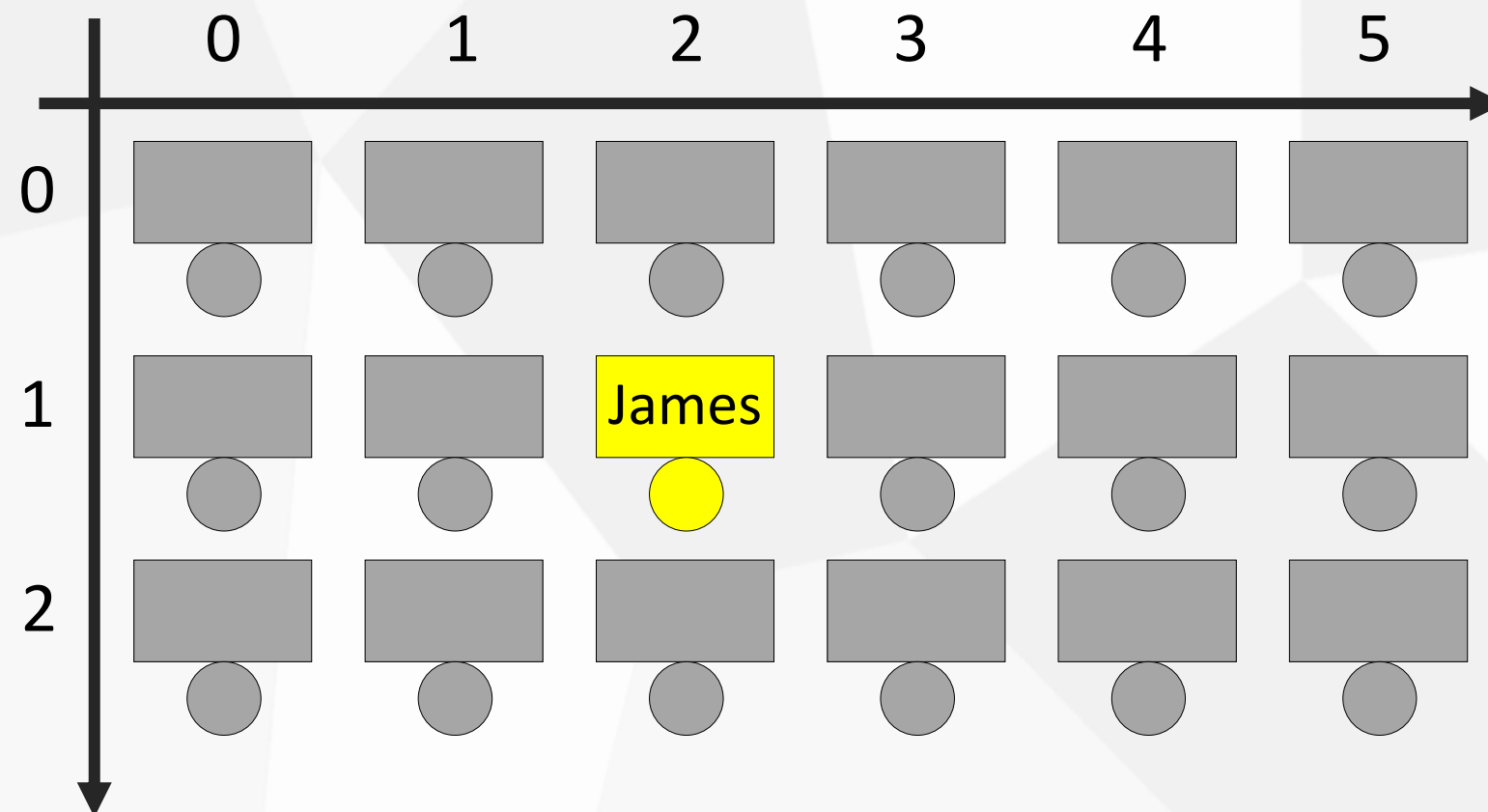
- Which of the following statements is NOT true?
 - An array is a list of data elements stored consecutively in the memory.
 - All the elements of an array are initially zero.
 - Array elements are accessed using an integer subscript.
 - Array elements can be modified using assignment statements.
 - Array elements must all be of the same type.

Quick Quiz about Arrays

- Which of the following statements is NOT true?
 - An array is a list of data elements stored consecutively in the memory.
 - ~~All the elements of an array are initially zero.~~
 - Array elements are accessed using an integer subscript.
 - Array elements can be modified using assignment statements.
 - Array elements must all be of the same type.

Two Dimensional Arrays

- How can we identify a student in a classroom?
- Classroom[1][2] is “James”.



Two Dimensional Arrays

- In C, **two-dimensional arrays can be constructed with two different subscripts**, and they are normally considered as: `[row_index][col_index]`
- A typical example can be given of the distance table between pairs of cities, how can we represent this as a 2D array in C?

	Aberdeen	Inverness	Glasgow	Edinburgh
Aberdeen	0	106	147	125
Inverness	106	0	173	157
Glasgow	147	173	0	48
Edinburgh	125	157	48	0

Two Dimensional Arrays

How to initialize a 2D array?

- Declare and initialize at the same time:

```
int mile[4][4]={{0,106,147,125},  
               {106,0,173,157},  
               {147,173,0,48},  
               {125,157,48,0}};
```

- The **row index can be left empty** as long as initialization is performed.
The compiler will determine the number of rows.

```
int mile[][4] =  
{0,106,147,125,106,0,173,157,147,173,0,48,125,157,48,0};
```

Two Dimensional Arrays

Access an element from a 2D array

- Row first: Using **array_name[i-1][j-1]**, we can access the element at the *i*-th row and *j*-th column of the 2D array.

```
int mile[4][4]={ {0,106,147,125},  
                 {106,0,173,157},  
                 {147,173,0,48},  
                 {125,157,48,0}};
```

```
printf("%d\n",mile[0][0]);  
printf("%d\n",mile[3][2]);
```

Nested Loops and Arrays

- Two-dimensional Arrays can be assigned by nested loops
- In nested loops, the whole interior loop will be executed for each iteration of the exterior loop.

```
int array[4][5];  
for(ii=0;ii<4;ii++)  
{  
    for(jj=0;jj<5;jj++)  
    {  
        array[ii][jj]=ii+jj;  
    }  
}
```


More Dimensions

- You can define arrays as many dimensions as you like;
- A two-dimensional array looks like a table;

- A three-dimensional array looks like a stack of tables:

float table[3][5][2];

- 3 nested loops would be required to process through all of the elements in this way.

Quick Quiz for Counting

- What will be printed out?
 - a). 31
 - b). 59
 - c). 90
 - d). 121
 - e). 151

```
int days[]={31,29,31,30,31,30,31,31,30,31,30,31};  
int i=0, sum=0;  
while(i<4)  
{  
    sum = sum + days[i++];  
}  
printf("sum=%d\n", sum)
```

Quick Quiz for Counting

- What will be printed out?
 - a). 31
 - b). 59
 - c). 90
 - d). 121
 - e). 151

```
int days[]={31,29,31,30,31,30,31,31,30,31,30,31};  
int i=0, sum=0;  
while(i<4)  
{  
    sum = sum + days[i++];  
}  
printf("sum=%d\n", sum)
```

Break

Introduction

What is a pointer?

- We have already learned how to declare and initialize variables like:

`int a=5;`

`char ch='A';`

- However, do we know where the PC stores these values and how it locates them?
- How can we find our items in a delivery station?



Introduction

What is a pointer?

- A pointer is a variable used to store the address of another variable.
 - It is similar to the **street address** of your home;

Pointer

c's address



a's address	a = 5
b's address	b = 4
c's address	c = -19
d's address	d

Introduction

What is a pointer?

- A pointer is a variable used to store the address of another variable.
 - It is similar to the **street address** of your home;
- Pointers are identified by the **type of variables** they are **pointing at**:
 - a **float** pointer must point at a **float** variable;
 - an **int** pointer must point at an **int** variable;
 - Internally, a pointer is stored as an **unsigned integer -> for address**;
 - However, you **cannot** use a pointer as you use for **int**;
 - For example, you **cannot multiply two pointers**.

Pointer Declaration

- Specify what type of variable the pointer is pointing at:

```
int *pAge; // pAge is pointing at an int.  
double *pHeight; // pHeight is pointing at a double.
```

- `pAge` and `pHeight` are pointers, and they store addresses;
- Q: Since `int` and `double` need different numbers of byte (4 and 8 respectively), do you think `pAge` and `pHeight` need different number of bytes?
- A: No. **They store the same thing.** —> Address.

Pointer Declaration

- Specify what type of variable the pointer is pointing at:

```
int *pAge; // pAge is pointing at an int.  
double *pHeight; // pHeight is pointing at a double.
```

- Q: If **they store the same thing**. —> Address. Then why we have to specify int and float separately?
- A: Because computer needs to use int/double's format to read this address.

Pointer Declaration

- You can also declare a pointer by using the following syntax:

```
int* a;
```

- In this way, it demonstrates that `int*` is **a distinct data type** from `int`.
- Which means, `int* a` is equivalent to `int *a`. You can use either way to declare a pointer.

Pointer Operators

The address operator '&' -> Get the address

- Placing the '&' operator in front of a variable will find the **address** of that **variable**;
- If `Age` is a variable, `&Age` is the **address** of that variable;
- The function `scanf()` makes use of the **& operator to determine where to store the value entered in the memory.**

```
int Age;  
scanf( "%d", &Age )
```

Pointer Operators

The address operator '**&**' -> Get the address

```
#include <stdio.h>

int main()
{
    int a,b;
    double c;
    char d,e;

    printf ("%p\n", &a);
    printf ("%p\n", &b);
    printf ("%p\n", &c);
    printf ("%p\n", &d);
    printf ("%p\n", &e);
    return 0;
}
```

- The format specifier '**%p**', used for printing the addresses.

Pointer Operators

The Dereference Operator **'*'** -> Get the value

- It **gets the value of the variable that the pointer is pointing at**;
- Use the * operator to find the value stored at an address;
 - **pAge** is the pointer which stores the address;
 - ***pAge** is the **value** stored at that address;
- If **pAge** **points** to an **integer**, ***pAge** is the **integer** value that is **being pointed at**.

Pointer Operations

Assignment

- Assign an address to a pointer.

```
int my_array[3]={10,20,30};  
int *pArray;  
pArray = &my_array[2]
```

- The **variable type** at that address should be compatible with the **pointer type**.

Pointer Operations

De-referencing

- Find the **value stored at the location the pointer is pointing at using the * operator**;
- ***pArray** gives the value that is stored at the address hold in **pArray**.

```
int my_array[3]={10,20,30};  
int *pArray;  
pArray=&my_array[2];  
printf("%d", *pArray) /* prints 30 */
```

Pointer Operations

De-referencing

```
int x=4, *p;
```

```
p = &x;
```

Address 4 bytes	Value	Variable
60ff10 - 60ff13	4	x

Address	Address of x	Variable
60ff14 - 60ff17	60ff10	p

```
*p = 10;
```

Address	Value	Variable
60ff10 - 60ff13	4	x

Address	Address of x	Variable
60ff14 - 60ff17	60ff10	p

Pointer Operators

De-referencing

```
char ch1 = 'G';
int age = 1175;
char ch2 = 'q';
```

```
char *pCh1;
pCh1 = &ch1;
```



```
/* Declare pointers and make them point to the variables */
char *pCh1 = &ch1; /* pCh1 stores the address of ch1 */
int *pAge = &age; /* pAge stores the address of age */
char *pCh2 = &ch2; /* pCh2 stores the address of ch2 */
```

```
/* Print the addresses stored in the pointers */
printf("%p\n", (void *)pCh1); /* Address of ch1 */
printf("%p\n", (void *)pAge); /* Address of age */
printf("%p\n", (void *)pCh2); /* Address of ch2 */
```

```
/* Dereference the pointers to access the values */
printf("%c\n", *pCh1); /* Value of ch1: 'G' */
printf("%d\n", *pAge); /* Value of age: 1175 */
printf("%c\n", *pCh2); /* Value of ch2: 'q' */
```

(void *): convert the current address to a common address regardless of its previous int/char or sth.

Address	52001	52002	52003	52004
Stored Value	G	1175	q	



Char takes up a memory cell: 1 byte.

Int takes 4 bytes.

Anything wrong here?

Pointer Operations

WARNING!!!

- **NEVER** dereference an uninitialized pointer:

```
int *ptr /* this is an uninitialized pointer */
```

- ptr contains a random value (which is always **not readable** by human);

```
*ptr = 5; /* this will generate a runtime error */
```

- This is **trying to store a value in an unknown location**;
- You don't know where the value is stored, and you will overwrite the data or code ...

Pointer & Array

- In C, the name of an array is actually a constant pointer to the first element of the array. Therefore, **the name of an array can be considered a constant pointer.**

- Example:

```
int arr[5];
```

- Here, `arr` is a pointer to the first int variable of the array, and `arr` is equivalent to `&arr[0]`.

Pointer & Array

- Example:

```
#include <stdio.h>

int main()
{
    char arr[3]={'A','@','Z'};

    printf("%p\n",arr); // array's name is a pointer
    printf("%p\n",&arr[0]);

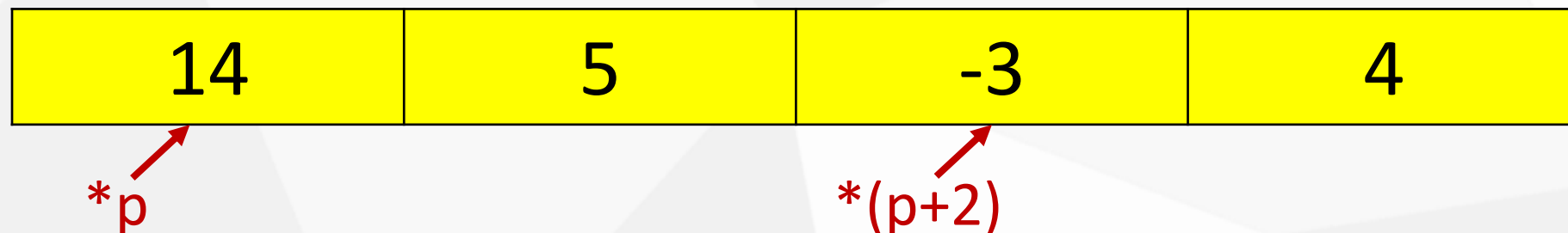
    printf("%c\n",*arr);
    printf("%c\n",arr[0]);

    // We can move the pointer to access other elements.
    printf("%c\n",*(arr+1));
    printf("%c\n",arr[1]);
    return 0;
}
```

Pointer & Array-small summary

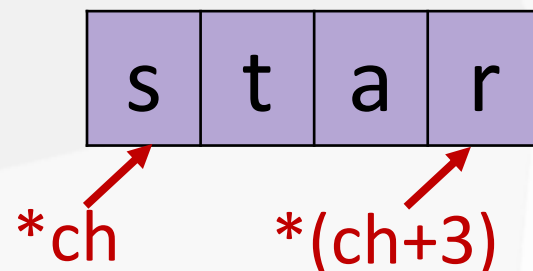
`int a[4]={14,5,-3,4};` // an int variable takes 4 bytes.

`int* p = a;` // You can use a pointer to access the array.

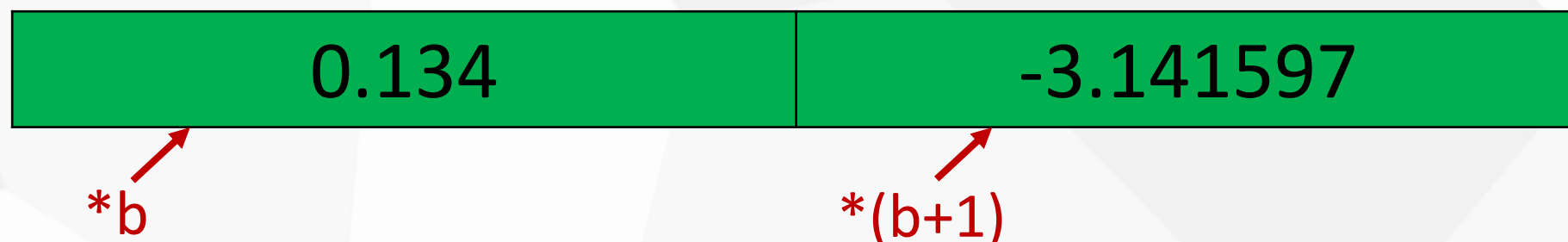


`char ch[4] = {'s','t','a','r'};` // a char variable takes 1 byte.

// The array name `ch` can be used as a pointer to the first element



`double b[2] = {0.134,-3.141597};` // Double variable takes 8 bytes



Pointer & Array

Moving the pointer with ++ and --

- **Incrementing (++)** and **Decrementing (--)** are possible with pointers;
- The value contained in the pointer is an **address**;
- **Incrementing** and **Decrementing** move the **number of bytes** for the **variable type**. -> **DONT FORGET IT! IT IS VERY IMPORTANT!**

```
int x[5]={4,2,3,8,9}, *p;    /* 4 bytes per interger value */
p=x;                        /* assigns &x[0] to p */
printf("%d, %p\n", *p, p);  /* prints 4, 0022FF1C */
p++;
printf("%d, %p\n", *p, p);  /* prints 2, 0022FF20 */
printf("%d, %p\n", *p, p);  /* prints 2, 0022FF20 */
printf("%p\n", &p);        /* prints the address of variable p, generally unknown */
printf("%p\n", (void *)&p);
```

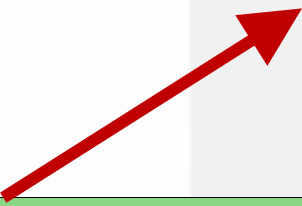
Pointer Operations

Moving the pointer with ++ and --

- Be careful with precedence when working with pointers;

***p++ and (*p)++ are not the same**

- * and ++ are both unary operators, the precedence is from right to left;



In C/C++, most operators' precedence is from left to right.
Only unary operators and assignment operators have precedence that is from right to left.

Pointer Operations

Moving the pointer with ++ and --

- Be careful with precedence when working with pointers;
***p++** and **(*p)++** are not the same
- * and ++ are both unary operators, the precedence is from right to left;
- ***p++** is equivalent to ***(p++)**, it will get the current address and then **increment the pointer**;
- **(*p)++** will get the value from the current address and **add 1 to that value**.

Pointer Operations

Pointer Subtraction: Differencing

- Differencing (or subtracting) two pointers can be used to determine **how many elements apart two elements are in an array**;

```
int x[5]={4,2,3,8,9}, *p1, *p2;  
p1=&x[0];  
p2=&x[2];  
printf("%d %d", *p1-*p2, p2-p1);
```

- What will be printed in this case?
 - The first **value** is **4-3=1**;
 - The second value is **2**, since the addresses are **8 bytes apart! (don't mess up here)**, which is equivalent to **2 integer** sizes.

Pointers and Arrays

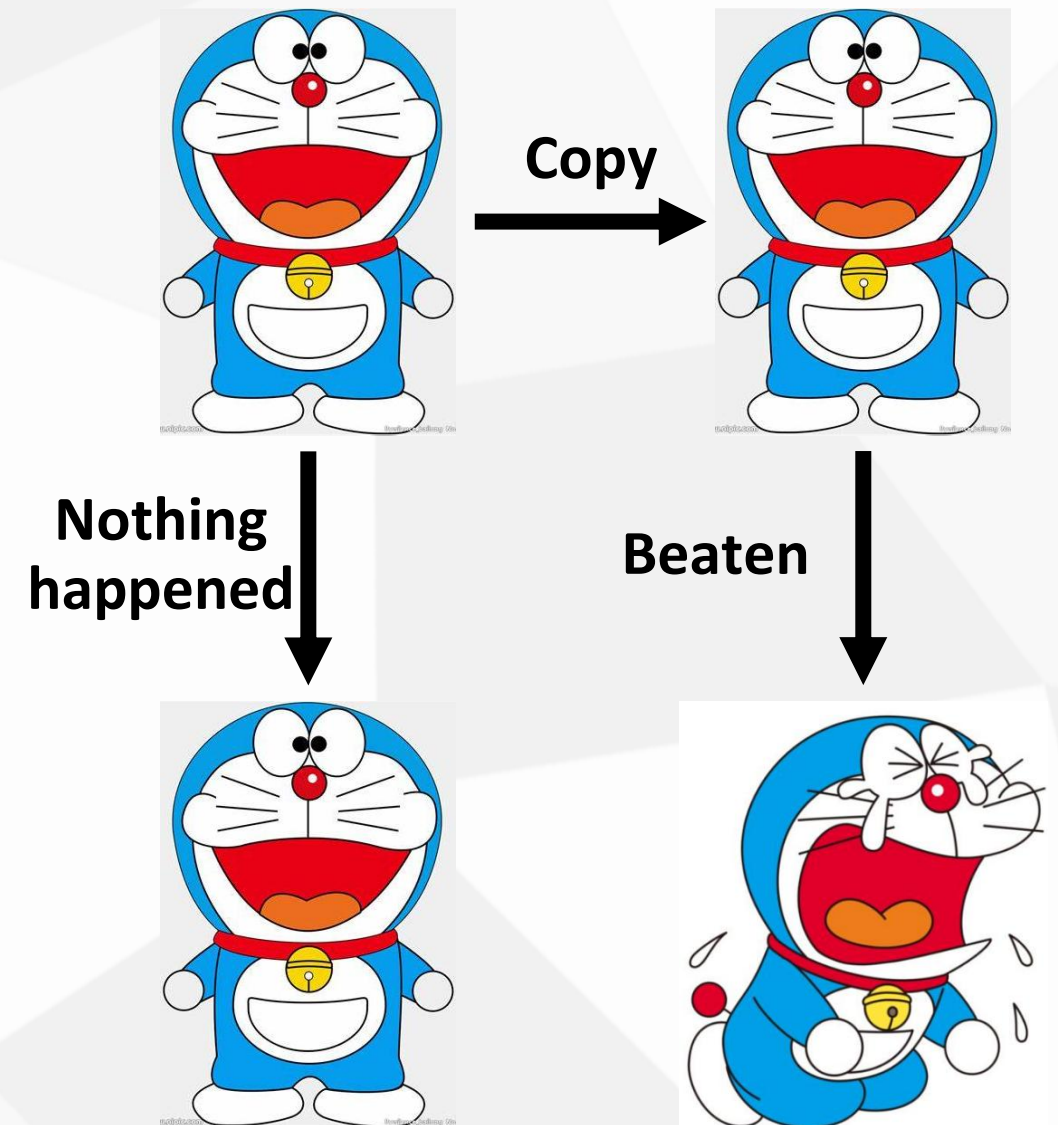
More on the relationship between Arrays and Pointers

- The name of an array is by itself **a pointer to (address of)** the first element of that array;
- So why do we still bother to define another pointer such as `int *ptr = &array[0];`?
 - It is because a pointer is a variable (so, `ptr++` is legal);
 - **But the array name is not a variable (so, `array++` is illegal).**

Call by Reference & Call by Value

Different scenarios

- Previously values stored in variables have been sent to functions with **calling by value**;
- This means that **the variable value itself cannot be changed inside the function**;
- Pointers allow access to a variable declared in one function to be **changed** within another function call **by reference**; (Important!)



Call by Reference & Call by Value

Example

```
#include <stdio.h>

int test(int);

int main()
{
    int y=10,z;
    z = test(y);
    printf("%d\n",y);
    return 0;
}

int test(int p){
    p = p*p;
    return p;
}
```

Call by Value

10

```
#include <stdio.h>

int test(int*);

int main()
{
    int y=10,z;
    z = test(&y);
    printf("%d\n",y);
    return 0;
}

int test(int* p){
    *p = (*p)*(*p);
    return *p;
}
```

int *p;
p= &y;

Call by Reference

100

Thanks